Implementing HDBSCAN: Insights, Limitations, and Applications on Real-World Datasets

Paul Chu
paulchuparis@gmail.com
ENS Paris-Saclay
Paris, France

Victor Guichard
guichardvictor@gmail.com
ENS Paris-Saclay
Paris, France

Eliott Vigier
eliott.vigier@gmail.com
ENS Paris-Saclay
Paris, France

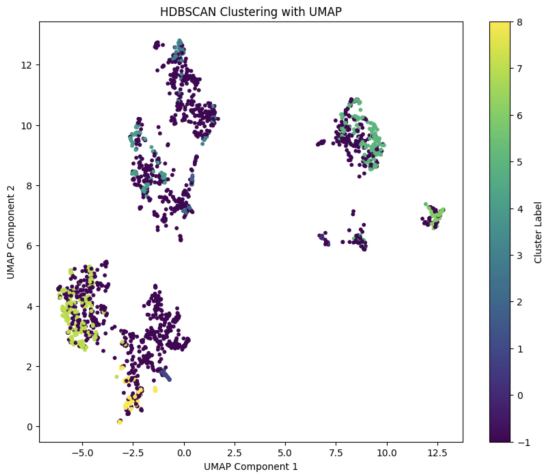

Figure 1: Visualization of HDBSCAN Clustering Results.

Abstract

Clustering is a fundamental unsupervised learning technique widely used for uncovering hidden structures in unlabeled datasets. Among clustering algorithms, HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise) has emerged as a robust solution for identifying complex cluster structures and handling noise in real-world datasets. Extending DBSCAN, HDBSCAN dynamically adapts to varying densities and produces high-quality clustering results without requiring precise parameter tuning.

In this study, we aim to demonstrate the implementation of HDBSCAN on a real-world dataset, providing insights into its practical applications, strengths, and limitations. Following a structured approach, we first present a theoretical foundation for clustering methods, including the mathematical principles and algorithms behind HDBSCAN. We then bridge the gap between theory and practice, using toy datasets to illustrate its functionality before applying it to a real-world case study. Our experimentation explores the algorithm's performance under various parameter settings, benchmark comparisons, and visualizations of the resulting clusters.

Finally, we discuss the limitations of HDBSCAN, particularly its sensitivity to high-dimensional data due to the curse of dimensionality, and explore potential solutions such as dimensionality reduction techniques. By benchmarking HDBSCAN against existing methods and analyzing its scalability, this paper offers a comprehensive evaluation of its practical applicability and identifies scenarios where its use is most effective. This work serves as both a guide and a critical analysis of HDBSCAN for clustering in diverse contexts.

Keywords

HDBSCAN

ACM Reference Format:
Paul Chu, Victor Guichard, and Eliott Vigier. 2024. Implementing HDBSCAN: Insights, Limitations, and Applications on Real-World Datasets. In . ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

1.1 Introduction to Clustering

Clustering is a core task in unsupervised learning, aimed at grouping unlabeled data points into clusters based on inherent similarities. Unlike supervised learning, where labeled data guides the model, clustering relies solely on the structure of the data itself. Formally, given a dataset $X = \{x_1, x_2, \ldots, x_n\}$, clustering seeks a partition $C = \{C_1, C_2, \ldots, C_k\}$, where each cluster C_i satisfies:

$$C_i \cap C_j = \emptyset \quad \forall i \neq j, \quad \bigcup_{i=1}^{k} C_i = X.$$

Conference'17, July 2017, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

The objective is to minimize intra-cluster distance while maximizing inter-cluster separation, ensuring that data points within the same cluster are similar, and points from different clusters are dissimilar.

1.2 Clustering Methods

Clustering methods can be broadly categorized into centroid-based, density-based, and hierarchical approaches:

1.2.1 K-Means Clustering. K-Means partitions the dataset into K clusters by minimizing the intra-cluster sum of squared distances. Formally, the optimization objective is:

$$\arg\min_{\mu_1,\ldots,\mu_K} \sum_{i=1}^{K} \sum_{x \in C_i} \|x - \mu_i\|^2,$$

where μ_i is the centroid of cluster C_i. While efficient, K-Means assumes clusters are spherical, equally sized, and requires K to be predefined.

1.2.2 DBSCAN (Density-Based Spatial Clustering of Applications with Noise). DBSCAN identifies clusters as dense regions of points separated by areas of lower density. Two key parameters define the clustering:

- ε: Maximum distance to consider two points as neighbors.
- minPts: Minimum number of points required to form a dense region.

Points are classified as *core*, *border*, or *noise* based on density:

$$\text{Core point: } |N_\varepsilon(x)| \geq \text{minPts}, \quad N_\varepsilon(x) = \{y \in X : \|x - y\| \leq \varepsilon\}.$$

DBSCAN excels at identifying irregularly shaped clusters but struggles when clusters vary significantly in density.

1.2.3 HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise). HDBSCAN[13] extends DBSCAN by building a hierarchy of clusters across varying density levels. It uses a *mutual reachability distance* to measure cluster connectivity:

$$d_{\text{mreach}}(x, y) = \max\{d_{\text{core}}(x), d_{\text{core}}(y), \|x - y\|\},$$

where $d_{\text{core}}(x)$ is the core distance, defined as:

$$d_{\text{core}}(x) = \min\{\varepsilon : |N_\varepsilon(x)| \geq \text{minPts}\}.$$

HDBSCAN constructs a Minimum Spanning Tree (MST) over the dataset using the mutual reachability distance and condenses the hierarchy to form clusters, improving performance on datasets with variable density and complex structures.

1.3 Mathematical Distances in Clustering

1.3.1 Euclidean Distance. The Euclidean distance between two points $x = (x_1, x_2, \ldots, x_d)$ and $y = (y_1, y_2, \ldots, y_d)$ in $\mathbb{R}^d$ is given by:

$$\|x - y\| = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2}.$$

1.3.2 Mutual Reachability Distance. Incorporating density into distance calculations, the mutual reachability distance defines the effective connectivity between points:

$$d_{\text{mreach}}(x, y) = \max\{d_{\text{core}}(x), d_{\text{core}}(y), \|x - y\|\}.$$

This metric ensures that denser regions are connected more tightly, allowing HDBSCAN to adapt to varying densities.

1.3.3 Dual-Tree Borůvka Algorithm. To efficiently compute the MST of the dataset under Euclidean distance, HDBSCAN leverages the *Dual-Tree Borůvka Algorithm*. The MST is the tree T minimizing the total edge weight:

$$T = \arg\min \sum_{(x,y) \in T} \|x - y\|.$$

This algorithm significantly reduces the computational complexity of MST construction, making HDBSCAN scalable to large datasets.

1.4 Existing Work

While K-Means remains widely used for its simplicity and speed, its limitations—such as sensitivity to initialization, fixed K, and inability to handle irregular cluster shapes—make it unsuitable for many real-world datasets. DBSCAN improves on these aspects by handling noise and irregular shapes but struggles with clusters of varying density. HDBSCAN[5] addresses these gaps by adapting to density variations and producing robust results even on synthetic, noisy, and complex datasets.

2 HDBSCAN Approach

2.1 Density Estimation

HDBSCAN begins by estimating the density of data points in the dataset. This involves identifying the (k)-nearest neighbors (KNN) for each point, where (k) is the minimum number of points required to form a cluster.

2.1.1 Density Estimation via k-Nearest Neighbors (KNN). The density around a point is inferred using its *core distance*, defined as the distance to its (k)-th nearest neighbor. A smaller core distance indicates a denser region, while a larger one suggests a sparser area. This method efficiently distinguishes dense clusters from sparse noise.

2.1.2 Refining density estimation using mutual reachability distance. After estimating the local density using the core distance, it is important to refine this measure to account for variations in density across the dataset. HDBSCAN achieves this refinement by introducing the *mutual reachability distance*, a distance metric that combines the core distances of two points with their original pairwise distance. The mutual reachability distance between two points a and b is defined as:

$$d_{\text{mreach}-k}(a, b) = \max\{\text{core}_k(a), \text{core}_k(b), d(a, b)\}$$

Here:

- $\text{core}_k(a)$ and $\text{core}_k(b)$ are the core distances of points a and b, respectively.
- $d(a, b)$ is the original distance between a and b.

This metric ensures that points in sparser regions are "pushed away" from other points, creating a clear distinction between dense regions and sparse regions.

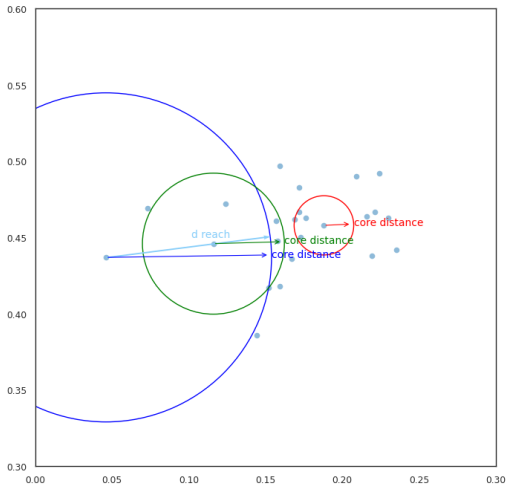

Figure 2: Mutual reachability distance example in two dimensions, where the mutual reachability distance is determined by the core distance of one of the two points.

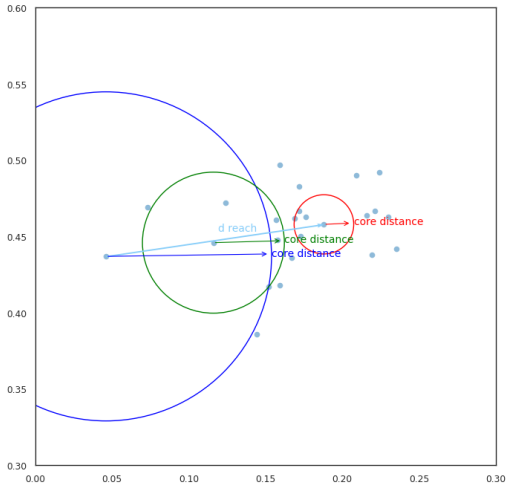

Figure 3: Mutual reachability distance example in two dimensions, where the mutual reachability distance is defined as the direct distance between the two points.

2.2 Building the Minimum Spanning Tree (MST)

Once we have computed the *mutual reachability distance* for all pairs of points, the next step is to start identifying clusters. To do this, we consider the dataset as a *weighted graph* where:

- Each data point represents a vertex.
- An edge exists between any two points, with the weight of the edge given by their mutual reachability distance.

2.2.1 Conceptual framework. To identify clusters, we imagine setting a threshold value for the edge weights. Initially, this threshold is very high, ensuring that all points are connected in a single large component. As we progressively lower the threshold, edges with weights above the threshold are removed, causing the graph to break into smaller connected components. These connected components represent clusters at different density levels, forming a hierarchy of clusters from fully connected (high threshold) to fully disconnected (low threshold).

While this approach is straightforward, it is computationally expensive. For a dataset with n points, there are $\binom{n}{2}$ edges, and checking connected components for each possible threshold is impractical for large datasets.

2.2.2 Efficiently building the Minimum Spanning Tree. Instead of analyzing all edges, we focus on finding a minimal set of edges that preserves the connectivity structure of the graph. This set must satisfy two conditions:

(1) Removing any edge from this set disconnects the graph.
(2) No edge outside this set has a lower weight that could reconnect the graph.

This specific set of edges is known as the *minimum spanning tree* (MST) of the graph. The MST ensures that we capture the essential structure of the graph while minimizing computational overhead.

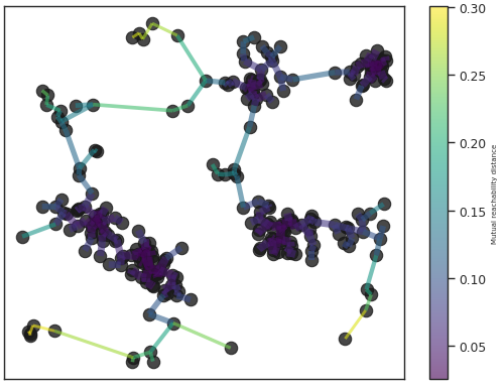

Figure 4: Example of the Minimum Spanning Tree

2.2.3 Using Prim's Algorithm. To construct the MST, we can use Prim's algorithm. The algorithm builds the tree iteratively:

(1) Start with an arbitrary vertex as the initial tree.
(2) Add the smallest edge (by weight) that connects a vertex in the tree to a vertex not yet in the tree.
(3) Repeat until all vertices are included in the tree.

The result is the minimum spanning tree based on the mutual reachability distance.

2.3 Building the Cluster Hierarchy

The next step is to transform the MST into a *hierarchy of connected components*. This hierarchy captures the merging of clusters at varying density levels and is key for identifying meaningful clusters in the dataset.

2.3.1 Constructing the hierarchy. The construction of the cluster hierarchy proceeds as follows:

(1) **Sort the edges of the MST:** Begin by sorting all the edges of the MST in increasing order of their weight (mutual reachability distance). This ensures that we process the smallest edges first, gradually merging denser regions before considering sparser ones.
(2) **Iterate through the edges:** For each edge, merge the two clusters connected by the edge. At the start, every data point is considered its own cluster. As edges are processed, clusters are combined.
(3) **Identify clusters using a union-find structure:** Efficiently track which clusters are being joined using a *union-find* data structure. This data structure allows for fast identification and merging of clusters during the iteration.

The result of this process is a dendrogram. Each level of the dendrogram corresponds to a threshold of mutual reachability distance, showing how clusters are merged as the threshold increases.

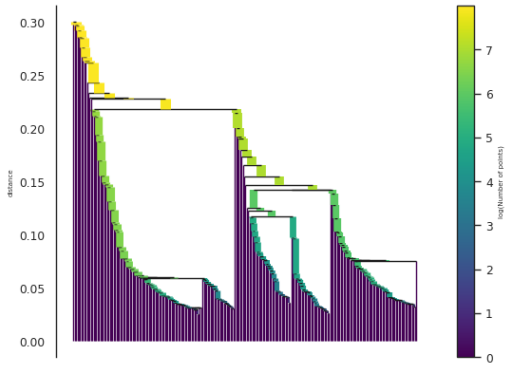

Figure 5: Example of a dendogram of a cluster hierarchy.

2.4 Condensing the Cluster Tree

After constructing the full cluster hierarchy, we need to condense this tree into a more interpretable structure. This step involves eliminating insignificant clusters and focusing on persistent clusters.

2.4.1 Condensation process. To condense the cluster tree, we proceed as follows:

(1) **Introduce a minimum cluster size:** Define a parameter, *minimum cluster size*, which specifies the smallest allowable size for a cluster. Clusters smaller than this size will be treated as noise or as points "falling out" of larger clusters.
(2) **Traverse the hierarchy:** Walk through the dendrogram from the root downward. At each cluster split:
   - If one of the resulting clusters has fewer points than the minimum cluster size, consider it as *points falling out of the cluster*. The larger cluster retains the identity of the parent cluster.
   - If both resulting clusters have sizes equal to or greater than the minimum cluster size, treat this as a *true cluster split* and allow the split to persist in the tree.
(3) **Record cluster dynamics:** For each cluster, record how its size changes over varying mutual reachability distances. This involves tracking the points that "fall out" of a cluster and noting the distance at which this occurs.

After processing the hierarchy, the tree becomes much smaller, containing only clusters that meet the minimum size criterion. Besides, each node has additional information about the cluster.

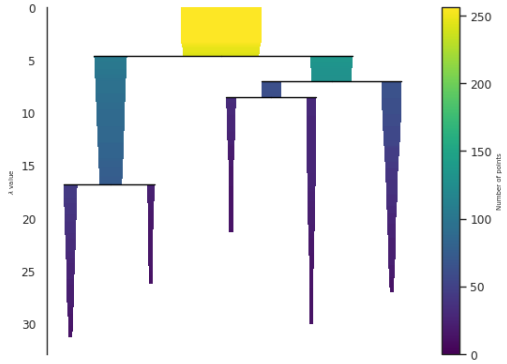

Figure 6: Example of a condensed cluster tree.

2.4.2 Visualizing the condensed tree. The condensed tree can be visualized as a modified dendrogram. In this version:

- The **width of each line** represents the number of points in the cluster.
- The **width decreases** along the length of the line as points fall out of the cluster.

2.5 Extracting Clusters

The final step in HDBSCAN is to extract meaningful clusters from the condensed cluster tree.

2.5.1 Cluster stability. To evaluate cluster persistence, we redefine distances in terms of their inverse:

$$\lambda = \frac{1}{\text{distance}}$$

For each cluster, we define:

- λ_{birth}: The λ value when the cluster first appears.
- λ_{death}: The λ value when the cluster ceases to exist.
- λ_p: The λ value at which a point p leaves the cluster.

Using these, the stability of a cluster is computed as:

$$\text{Stability} = \sum_{p \in \text{cluster}} (\lambda_p - \lambda_{\text{birth}})$$

This measures the cluster's persistence as the area under its curve in the condensed dendrogram.

2.5.2 Cluster selection algorithm. The algorithm for selecting clusters is as follows:

(1) Declare all **leaf nodes** in the tree as selected clusters.
(2) Traverse the tree **bottom-up**:
   - If the cluster's stability is greater than the sum of its child clusters' stabilities:

  - Declare it as a **selected cluster**.
  - Unselect all its descendants.
- Otherwise, set the cluster's stability as the sum of its child clusters' stabilities.

(3) At the root node, finalize the current set of selected clusters as the flat clustering.

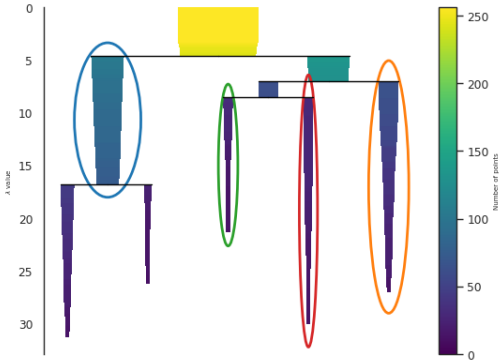

Figure 7: The chosen closers are circled.

2.5.3 *Assigning labels and membership strength.*

- **Cluster Labels:** Assign each point in a selected cluster a unique label.
- **Noise Points:** Points not in any selected cluster are labeled as noise (-1).
- **Membership Strength:** Normalize λ_p values for each point in a cluster to $[0, 1]$, providing a probabilistic measure of cluster membership.

3 Implementation and Experiments

3.1 Toy Example

To generate images in Section 2, we generated a noise pattern using the fractal Brownian motion which merges multiple layers of perlin noise together. From the 2d noise map, we can sample points based on the noise value as in figure 8.

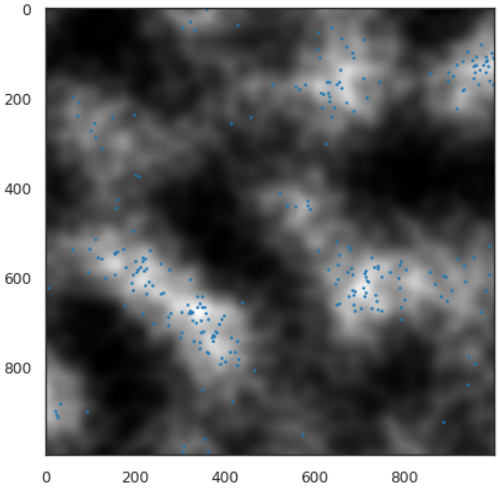

Figure 8: fBm point sampling

In this section, we will use self-generated data to experiment on the hyper parameters of HDBSCAN.

3.2 Hyper Parameters

The two main hyper parameters of HDBSCAN are *min_cluster_size* and *min_samples*. The first one determines how the cluster tree is condensed, while the latter defines the core distance. Theoretically, large values of *min_cluster_size* should lead to fewer but larger clusters. As *min_samples* is indirectly linked to the density estimation, a large value will make the algorithm classify more points as noise, as it requires denser clusters.

Our experiment on the same data shows it, the results are shown in figure 9 and figure 10.

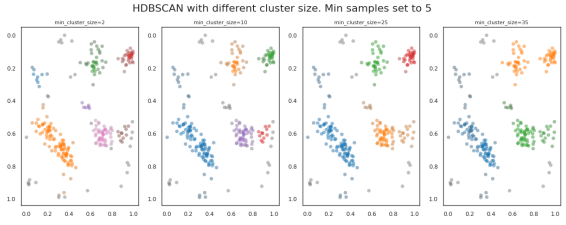

Figure 9: HDBSCAN results on our generated dataset with different values of *min_cluster_size*.

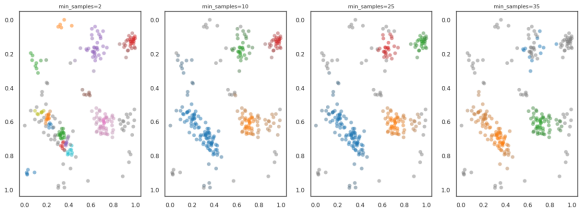

Figure 10: HDBSCAN results on our generated dataset with different values of *min_samples*, with *min_samples* set at 5.

3.3 Performance

We evaluated the performance of HDBSCAN against other clustering algorithms empirically, as both the implementation and the underlying algorithm can impact performance. The time required to fit each algorithm was measured across increasing dataset sizes, with results presented in Figure 11.

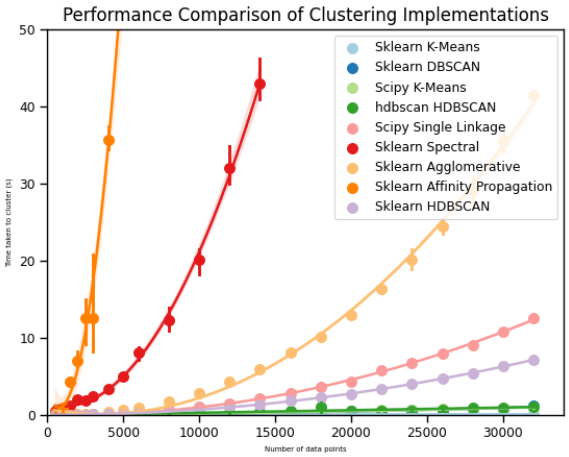

Figure 11: Performance comparison of various clustering algorithms.

We filtered out the slowest algorithms for better clarity. In particular, the HDBSCAN implementations in scikit-learn [15] and the hdbscan library exhibit significant differences in performance scaling.

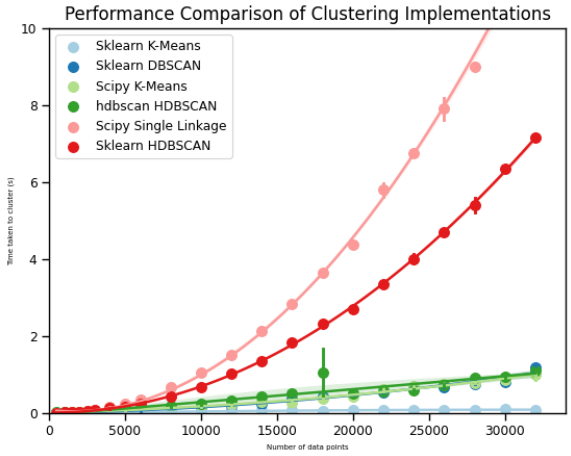

Figure 12: Performance comparison after filtering slower algorithms.

After further filtering, we observed that the remaining algorithms scale similarly, with K-means consistently outperforming others in terms of speed. The filtered results are shown in Figures 12 and 13.

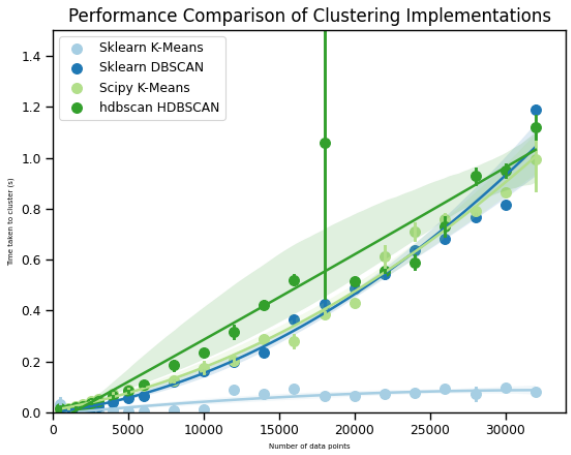

Figure 13: Final performance comparison of selected clustering algorithms.

3.4 Real World Case Study

To showcase the application of the HDBSCAN algorithm we decided to do an analysis of textual data from the Bluesky social media. Over 2M tweets were downloaded, and 32,000 were randomly sampled to keep the experiment small.

3.4.1 Pipeline of the analysis. To analyze Bluesky "tweets," multiple algorithms were employed, combining dimensionality reduction, clustering, and topic modeling. First each text were fed to mxbai-embed-large-v1 [10], [11], a large language model, to generate high-dimensional (1024-dimensional) embeddings of the tweets. These

embeddings captured the semantic nuances of the data, serving as a rich input for clustering.

To make the embeddings more appropriate to clustering, we used Uniform Manifold Approximation and Projection (UMAP) [14] to reduce the embeddings to a lower-dimensional space while preserving meaningful structures.

The reduced embeddings were then clustered using the HDBSCAN algorithms, and for each branch of the dendrogram a topic modeling approach was used, in our experiment the first few words based on their TF-IDF score.

This pipeline allows us to discover topics discussed in the social media while having no prior on the data.

3.4.2 Results. HDBSCAN allows us to discover many different topics. Figures 14 and 15 were done by reducing the dimensions of the embedding to 2 so that they can be visualize as an image, clusters are colored based on their topics such as *(Youtube, Podcast, Music), (Trump, Election, Mexico), (Russia, Ukraine, War), (Food, Pizza, Thanksgiving)* or *(Weather, Forecast, November)*. Interestingly, as the model used for embeddings was not performing well in multi languages, we observed some large cluster based on the language (Spanish, French, etc.).

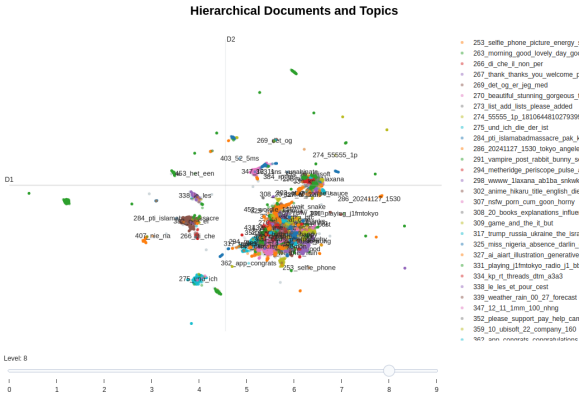

Figure 14: Clustering results of 32k tweets from the Bluesky social media. We observe that many different topics are found by our clustering pipeline.

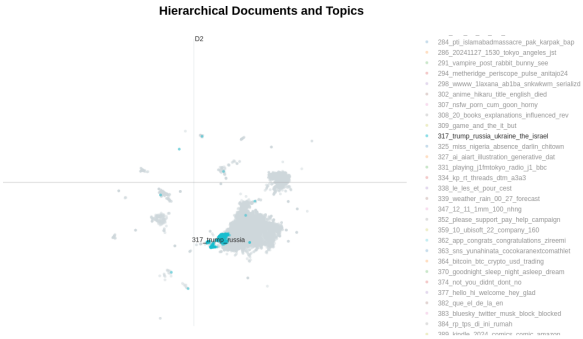

Figure 15: Clustering results of 32k tweets from the Bluesky social media. Points from a similar topics are condensed together forming a cluster in this case *Trump, Ukraine, ...*

The intertopic distance map can be seen in figure 16.

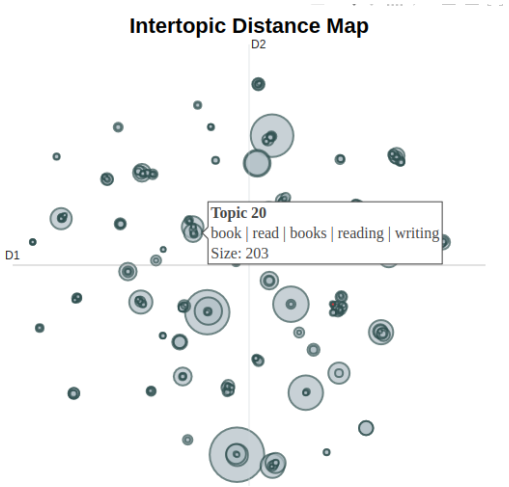

Figure 16: Inter-topic distance map allows us to more easily identify the topics and their distance between each others.

The use of HDBSCAN for such scenarios has multiple advantages over other algorithms:

- Shape: Models like HDBSCAN assume that clusters can have different shapes and forms. As a result, using a centroid-based technique to model the topic representations would not be beneficial since the centroid is not always representative of these types of clusters.
- Noise Identification: Unlike DBSCAN, which can struggle with complex or high-dimensional data, HDBSCAN efficiently identifies outliers as noise. This ensures that clusters are composed of data points with meaningful connections, avoiding distortions from less-relevant data.
- No Fixed Number of Clusters: HDBSCAN does not require a pre-specified number of clusters, unlike K-Means. This

makes it ideal for exploratory analysis of data where the true number of clusters is unknown.
- Scalability: HDBSCAN is computationally efficient for large datasets, handling the scale of the Bluesky dataset (originally 2M but reduced to 32,000 tweets in our experiment) with ease while preserving good performance.

4 Limitations of HDBSCAN

HDBSCAN has notable limitations that make its performance highly dependent on specific dataset characteristics. While it excels at detecting clusters with arbitrary shapes and varying densities, it struggles in scenarios involving spherical clusters, high-dimensional data, uniform-density datasets, or significant noise. Furthermore, the algorithm's reliance on hyperparameters such as *min_cluster_size* and *min_samples* requires careful tuning, making it sensitive to suboptimal parameter selection. Below, we analyze these limitations in detail using the results of our experiments, organized by dataset characteristics:

Dataset	Method	ARI	Silhouette
Breast Cancer	GMM[8]	0.774016	0.314489
	KMeans	0.653625	0.343382
	MeanShift[4]	0.269276	0.168140
	HDBSCAN	0.118554	-0.047853
Iris	GMM[8]	0.903874	0.374165
	KMeans	0.645147	0.456535
	HDBSCAN	0.563751	0.494863
	MeanShift[4]	0.536264	0.353130
MNIST	GMM[8]	0.282834	0.012530
	KMeans	0.249380	0.017750
	HDBSCAN	0.045981	-0.170825
	MeanShift[4]	0.000724	0.137828
Wine	KMeans	0.863599	0.282837
	GMM[8]	0.863599	0.282837
	MeanShift[4]	0.376718	0.200543
	HDBSCAN	0.265957	0.098521

Table 1: Clustering performance comparison using ARI[6] and Silhouette[9] scores across datasets and methods.

- For noise sensitivity, refer to the analysis in **A.1** and Figure 17.
- For spherical cluster scenarios, see **A.2** and Figure 18.
- For challenges in high-dimensional data, consult **A.3** and Figure 20.
- For uniform density datasets, examine **A.4** and Figure 21.

The corresponding detailed analysis and figures for these experiments are presented in the annex section.

5 Extension

5.1 Adapting HDBSCAN to Address Limitations

Enhancements to HDBSCAN include dynamic density thresholds and preprocessing (e.g., Isolation Forest) for noise handling, integration with centroid-based methods for spherical clusters, and dimensionality reduction (e.g., PCA, UMAP) for high-dimensional data. Kernel Density Estimation [7] and hybrid approaches improve uniform-density clustering, while automated parameter tuning and scalable implementations (e.g., ANN, distributed systems) increase efficiency and adaptability across diverse datasets.

5.2 Soft Clustering for HDBSCAN

Soft clustering enhances HDBSCAN's utility in scenarios with overlapping clusters or ambiguous boundaries, such as bioinformatics and natural language processing. Figures 22 and 23, included in **Section B.1** of the annex, illustrate clustering on the Wine dataset before and after optimization. Figure 22 shows initial membership probabilities from HDBSCAN and GMM, highlighting areas of uncertainty and overlap. After optimization, Figure 23 demonstrates improved handling of noisy points and irregular clusters by HDBSCAN, alongside GMM's refined Gaussian clusters with compact memberships. These results underscore the complementary strengths of the methods and the promise of soft clustering in extending HDBSCAN's effectiveness across diverse tasks.

5.3 Outlier Detection with HDBSCAN

HDBSCAN's ability to label sparse points as noise makes it useful for outlier detection but lacks quantified scoring or ranking. Adding a density-based outlier scoring mechanism and integrating models like Isolation Forest [12] could enhance anomaly identification. These improvements would broaden HDBSCAN's applications in fraud detection, cybersecurity, and medical diagnostics by enabling precise and actionable insights into critical outliers.

6 Conclusion

This report shows the versatility and effectiveness of HDBSCAN in addressing complex clustering problems, particularly with real-world datasets like the 32,000 Bluesky tweets analyzed in our case study. The algorithm stands out for its ability to adapt to different density patterns and detect noise without requiring extensive parameter adjustments, making it suitable for diverse and unstructured data.

However, our analysis also revealed some limitations, such as HDBSCAN's sensitivity to high-dimensional datasets, where its performance can decrease due to the curse of dimensionality. Techniques like dimensionality reduction and careful parameter selection can help overcome these issues. In comparison with other algorithms, HDBSCAN proved to be flexible and effective at managing noise, but it was not always the fastest or simplest option.

In summary, HDBSCAN is a reliable and robust tool for clustering real-world data, as long as its limitations are taken into account. This study highlights its practical benefits while offering useful recommendations for researchers and practitioners seeking to use HDBSCAN effectively.

References

[1] Stefan Aeberhard and M. Forina. 1992. Wine. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5PC7J.

[2] Li Deng. 2012. The mnist database of handwritten digit images for machine learning research. *IEEE Signal Processing Magazine* 29, 6 (2012), 141–142.

[3] R. A. Fisher. 1936. Iris. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C56C76.

[4] K. Fukunaga and L. Hostetler. 1975. The estimation of the gradient of a density function, with applications in pattern recognition. *IEEE Transactions on Information Theory* 21, 1 (1975), 32–40. https://doi.org/10.1109/TIT.1975.1055330

[5] John Healy. 2018. HDBSCAN, Fast Density Based Clustering, the How and the Why. YouTube video. https://www.youtube.com/watch?v=dGsxd67IFiU.

[6] Scikit learn Developers. 2024. Adjusted Rand Index, sklearn.metrics.adjusted_rand_score. Online; accessed December 4, 2024. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.adjusted_rand_score.html.

[7] Scikit learn Developers. 2024. Density Estimation. Online; accessed December 4, 2024. https://scikit-learn.org/1.5/modules/density.html.

[8] Scikit learn developers. 2024. *Gaussian Mixture Models — Scikit-learn 1.5 Documentation.* https://scikit-learn.org/1.5/modules/mixture.html.

[9] Scikit learn Developers. 2024. Silhouette Score, sklearn.metrics.silhouette_score. Online; accessed December 4, 2024. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.silhouette_score.html.

[10] Sean Lee, Aamir Shakir, Darius Koenig, and Julius Lipp. 2024. *Open Source Strikes Bread - New Fluffy Embeddings Model.* https://www.mixedbread.ai/blog/mxbai-embed-large-v1

[11] Xianming Li and Jing Li. 2023. AnglE-optimized Text Embeddings. *arXiv preprint arXiv:2309.12871* (2023).

[12] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. 2008. Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining*. IEEE, IEEE, 413–422.

[13] Leland McInnes. 2016. High Quality, High Performance Clustering with HDBSCAN. YouTube video. https://www.youtube.com/watch?v=AgPQ76RIi6A.

[14] Leland McInnes, John Healy, and James Melville. 2020. UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. arXiv:1802.03426 [stat.ML] https://arxiv.org/abs/1802.03426

[15] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

[16] Matjaz Zwitter and Milan Soklic. 1988. Breast Cancer. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C51P4M.

A Limitations of HDBSCAN

A.1 Noise Sensitivity: Breast Cancer Dataset

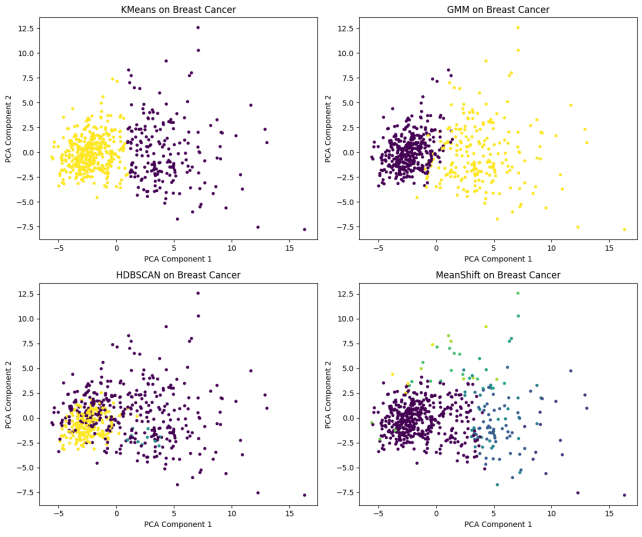

Figure 17: Comparison of clustering methods on the Breast Cancer dataset visualized using PCA-reduced data

HDBSCAN relies heavily on density estimation, which makes it highly sensitive to noise. In the Breast Cancer dataset [16], the presence of sparse or noisy data points led to a significant proportion of points being labeled as noise, resulting in poor clustering performance. As seen in the results, HDBSCAN achieved an ARI of only 0.118554 and a negative Silhouette score of -0.047853, indicating poor alignment with ground truth labels and overlapping or misclassified clusters. In contrast, GMM and KMeans, which are less affected by noise, achieved ARIs of 0.774016 and 0.653625, respectively. These results demonstrate HDBSCAN's susceptibility to noise in datasets where key features are sparse or overlap significantly.

A.2 Spherical Clusters: Iris Dataset

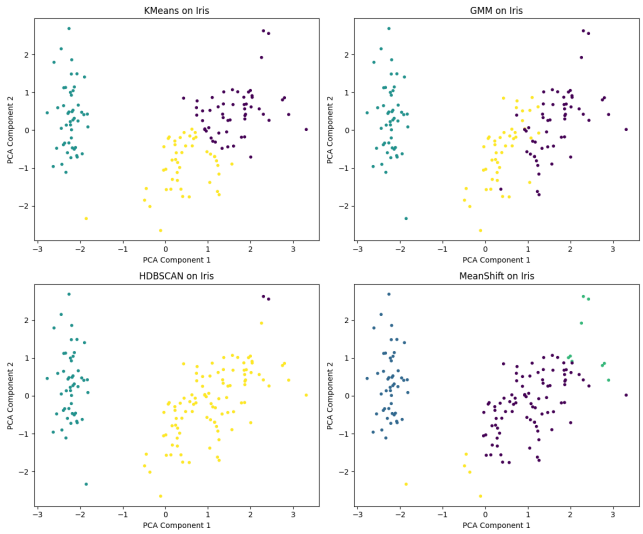

Figure 18: Clustering results on the Iris dataset visualized using PCA

While HDBSCAN excels at detecting arbitrarily shaped clusters, it struggles when clusters are compact, spherical, and well-separated. This limitation is evident in the Iris dataset [3], where HDBSCAN achieved an ARI of 0.563751, significantly lower than GMM (ARI: 0.903874) and KMeans (ARI: 0.645147). Interestingly, HDBSCAN recorded the highest Silhouette score (0.494863) among all methods, indicating compact and well-separated clusters. However, this high Silhouette score is misleading, as it reflects cluster compactness rather than alignment with the true labels. The tendency of HDBSCAN to merge spherical clusters of similar density further contributed to its underperformance in this dataset.

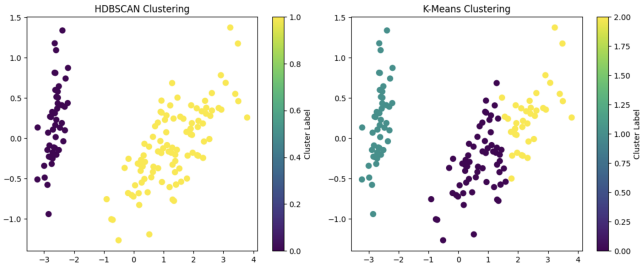

Figure 19: Comparison of clusters using HDBSCAN and K-Means.

K-Means assumes spherical, evenly distributed clusters, performing well on simple, well-separated datasets but struggling with irregular shapes or varying densities. HDBSCAN excels in detecting irregular clusters and handling noise but is less effective with spherical clusters or noise-free datasets.

A.3 High-Dimensional Data: MNIST Dataset

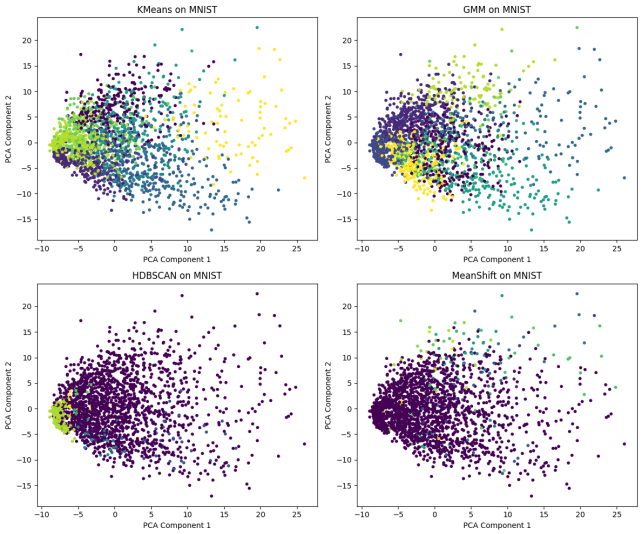

Figure 20: Clustering results on the MNIST dataset visualized using PCA

HDBSCAN's density-based approach is particularly challenged by the "curse of dimensionality," where distances between points in high-dimensional spaces lose their discriminatory power. This renders density estimation unreliable and diminishes the algorithm's ability to form meaningful clusters. In our experiments on the MNIST dataset [2], HDBSCAN achieved a near-random ARI of 0.045981 and a negative Silhouette score of -0.170825, indicating poorly defined clusters and significant misclassification.

In contrast, GMM (ARI: 0.282834) and KMeans (ARI: 0.249380) performed slightly better, albeit still limited due to the dataset's complexity. These results underscore the importance of dimensionality reduction techniques in high-dimensional datasets. Methods such as PCA and UMAP can mitigate these challenges by reducing the number of dimensions while preserving key relationships among data points. PCA simplifies the feature space by projecting data onto directions of maximum variance, while UMAP preserves both local and global structures, making clusters more distinct and separable. Applying these techniques before clustering could significantly improve HDBSCAN's performance in high-dimensional datasets like MNIST.

A.4 Uniform Density: Wine Dataset

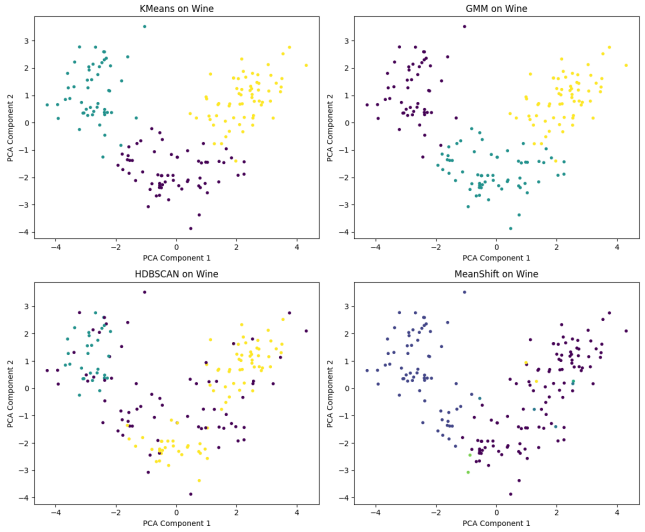

Figure 21: Clustering results on the Wine dataset visualized using PCA

HDBSCAN relies on variations in density to identify clusters. In the Wine dataset [1], where clusters are distributed uniformly, the algorithm struggled to differentiate clusters effectively. HDBSCAN achieved an ARI of 0.265957 and a Silhouette score of 0.098521, both significantly lower than those of KMeans and GMM, which achieved ARIs of 0.863599. These results suggest that HDBSCAN's performance is limited when density variations are minimal, as it fails to establish meaningful cluster boundaries. Methods such as KMeans and GMM, which partition data based on distance metrics or Gaussian assumptions, are better suited for uniformly dense datasets.

B Extension

B.1 Soft Clustering for HDBSCAN

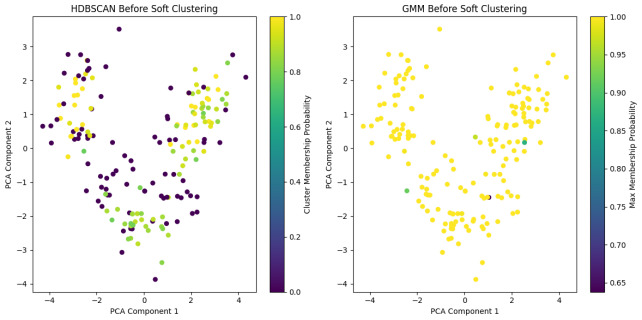

Figure 22: Clustering results before optimizing for the Wine dataset using HDBSCAN and GMM

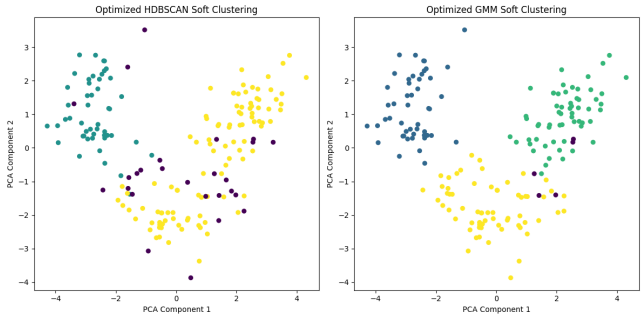

Figure 23: Optimized Soft clustering results for the Wine dataset using HDBSCAN and GMM.